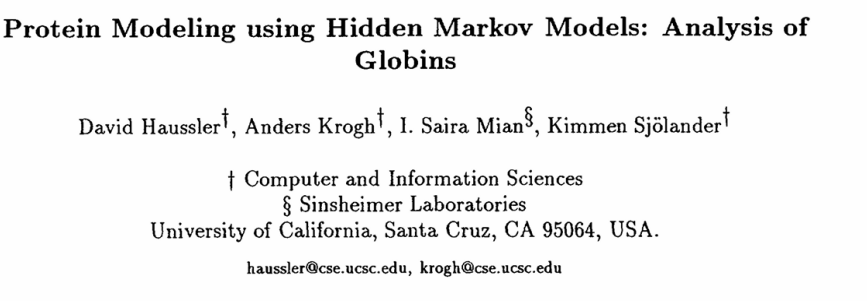
基于隐马尔可夫模型的多序列比对算法的 python 实现

熊婧茹 2300012141

1. Profile Hidden Markov Models (HMMs)简介

Profile Hidden Markov Model（profile HMM）是一种基于概率图模型的统计方法，被广泛应用于蛋白质和核酸序列的建模与比对任务。它最早由 Haussler 等人在 1993 年夏威夷系统科学国际会议（Hawaii International Conference on System Sciences）上提出，用于蛋白质家族 globin 的建模。此后，Krogh 等人在 1994 年《Journal of Molecular Biology》上发表了 "Hidden Markov models in computational biology: Applications to protein modeling"一文，进一步系统化和推广了该模型。现在它成为生物信息学中使用最广泛的统计模型之一，应用范围极广，包括蛋白质结构与功能预测、蛋白质家族分类与注释、新蛋白质归类至已有的蛋白质家族与超家族、宏基因组学以及多序列比对等。目前主流工具如 HMMER 和 SAM 都是基于 profile HMM 原理构建的，其中 HMMER 被广泛用于 Pfam 数据库的构建与比对。



Protein Modeling using Hidden Markov Models: Analysis of Globins

David Haussler†, Anders Krogh†, I. Saira Mian§, Kimmen Sjölander†

† Computer and Information Sciences
§ Sinsheimer Laboratories
University of California, Santa Cruz, CA 95064, USA.

haussler@cse.ucsc.edu, krogh@cse.ucsc.edu

图 1 1993 年 profile HMM 的最早提出

2. Profile HMM 用于多序列比对的基本流程

Profile HMM 是一种基于隐马尔可夫模型的统计工具，用于建模多序列比对中反映演化特征的模式。它的使用流程通常包括两个步骤：

第一步，对一组相关的分子序列构建一个 profile HMM，通常基于已有的多序列比对结果（MSA），提取各列的保守性信息并定义 Match、Insert 和 Delete 状态，估计转移概率与发射概率，从而形成完整模型。

第二步，将新的待分析序列输入该模型，使用 Viterbi 等算法进行比对，计算其在模型中的最可能路径及对应的 log-likelihood，以判断其是否属于该序列家族。

一个标准的 profile HMM 包含以下组成部分：

- Match 状态（M）：表示保守位置，具有发射概率；
- Insert 状态（I）：建模序列中插入片段，也具有发射概率；
- Delete 状态（D）：表示某些位置的缺失，不发射字符；
- 转移概率：定义状态之间的跳转可能性；
- 发射概率：描述各状态发出碱基/氨基酸的概率分布。

Profile HMM 通过建模序列的"结构+变异"特征，广泛应用于序列识别、家族分类和功能注释等领域。

在作业的第一部分，我们构建了一个完整的 profile HMM，通过读取已有的多序列比对结果 MSA，进行模型训练，统计每一列的保守性信息，构建 Match/Insert/Delete 状态，使用 Baum-Welch 算法估计各状态之间的转移概率和发射概率，构建一个 profile HMM；然后输入一个待分析的新序列，根据我们构建的 profile HMM 对齐，使用 Viterbi 算法计算最可能的路径，输出对齐后的序列比对结果。但是我们发现这部分代码的时间复杂度过高，对于真实的 MSA 结果无法很好的在短时间内实现序列的对齐，因此我们通过查找资料，重新写了第二部分的方法来进行序列对齐。

在本作业的第二部分中，我们的程序实现了以下功能：输入已有的多序列比对结果 MSA，进行模型训练，并构建 profile HMM；然后输入待分析新序列，进行对齐并使用 Viterbi 算法计算最可能的路径的 log-likelihood，并判断该序列是否可能属于原始 MSA 所代表的序列家族。相比于第一部分的代码，第二部分的代码进行了大幅优化。第一部分使用的是 Baum-Welch 算法来估计转移概率和发射概率，这是一个迭代过程，需要多轮遍历所有序列和状态，计算开销大，无法在短时间内完成大规模比对。第二部分则是通过直接计数和 Laplace 平滑估计转移概率和发射概率，单次遍历 MSA 即可完成训练，极大地降低了训练时间复杂度。此外，我们对 Viterbi 算法的实现进行了加速，使用了 Numba 的 JIT 编译（@jit(nopython=True)）将核心函数编译为本地机器码，极大提升了运行效率，使得我们的程序可以很快输出测试序列的 log-likelihood 得分，远快于第一部分使用 Baum-Welch 多次迭代。

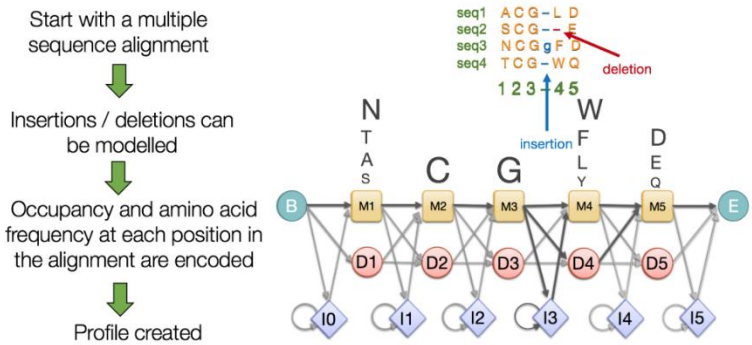


图 1　Profile HMM 建模流程

3. Profile HMM 用于多序列比对的 python 实现

第一部分的具体代码详见文件夹 hmm_profile_part1 中的 ProfileHMM_complete.py 文件，运行方式见 README.txt 文件的说明。

第二部分的具体代码详见文件夹 hmm_profile_part2 中的 ProfileHMM.py 文件及 main.py 文件，运行方式见 README.txt 文件的说明。

4. 输入数据及获取方法

第一部分的测试数据是由 GPT 生成的一组短的 DNA-MSA 样例，由于当前算法的时间复杂度较高，在有限运行时间内难以处理较长的 DNA-MSA，因此选取短序列数据作为测试输入以保证程序可以顺利执行并输出结果。测试数据可以在 hmm_profile_part1 文件夹中的 testcase/in/input.txt 里面找到。

第二部分测试数据来自 Rfam 数据库的 RNA 家族 RF00003（U1 spliceosomal RNA），这是一个高度保守的核内小 RNA 家族，广泛存在于多个物种中，参与剪接体复合物的组装，获取的原始地址是 https://rfam.org/family/RF00003#tabview=tab2。其原始 fasta 文件可以在

hmm_profile_part2 文件夹中的“train.txt”文件中找到，而用来作为新序列对齐的 hmm_profile_part2 文件夹中的“test.txt”则是选取了这个家族中的几条 RNA 序列，分别是 V00590，X01093 及 X14419。

5. 运行结果呈现与分析

（1）第一部分运行结果：

输入测试数据：

input.txt

```
5 3
ACGT-CACGA
ACG--CACGG
A-GTCCACGC
ACGTT-ACGA
ACGTTCACGT
GATGTCAT
```

输出对齐结果：

```
(base) celina@bogon 3 % python /Users/celina/Desktop/3/hmm_profile.py
testcase
GATGTCAT    -G-ATGTCAT
```

```
5 3
ACGT-CACGA
ACG--CACGG
A-GTCCACGC
ACGTT-ACGA
ACGTTCACGT
GATGTCAT
-G-ATGTCAT
```

我们可以看到该模型是可以很好的实现新序列的对齐的。

（2）第二部分运行结果：

输入 MSA 数据：

train.txt

```
>X06810.1/261-421
AUACUUACCUGGACGG-GGUCA-AUGG---AUGAUCAA-UAAG-GUCCA-UGGCCU---AGG-GAAGUAACCUCCAUU-----GCACUU-AGGAGG-GGUGCUUU---------CCUA-AGGUCUGCCCAA---GUGG--CAG-AGCCU-
ACGUCAUAAUUUGUGGUAG--UGGGGG--CUUGCGUU--CGCGCAGCCCCUUC
>X14417.1/177-340
AUACUUACCUGGACGG-GGUCA-AUGG---GUAAUCAA-GAAG-UUCCA-UGGCCU---AGG-UUGGUGACCUCCAUU-----GCACUA-AGGAGG-GGUGCUUG---------CCUA-AGGUCGACCCAA---GUGG--UUG-AGCCU-
ACGUCAUAAUUUGUUGUUGCAGAGGGG-GCCUGUGUU--CGCGCAGCCCCUAC
>Z11883.1/3441-3603
UUACUG--GUGGGUGGCGGAGA-AGAGAUCGUGACAUC-GUAG-ACCUA-UGGCUU---AGA-UUGAUGACCUCCAUU-----GCACUU-UGGAGG-GGUGCCUG---------CCUAAAGUCUG-CCCAA---GUGG--UCG-AGCCU-
ACACUAUAAUUUGUGAUAG---UGGGG-GCUUGCGUU--CGCGCAGCCCCUAG
>X14419.1/178-338
AUACUUACCUGGACGG-GGUCA-AUGG---GCGAUCAU-GAAC-ACCCA-UGGCCU---AGG-UUGGUGACCAUCAUU-----GCACUU-UGAAGG-GGUGCCCG---------CCUA-AGGUCGGCCCAA---GUGG--UCG-AGCCU-
ACGUCAUAAUUUGUUGCUG---AGGGG-GCCUGCGUU--CGCGCGGCCCCUGC
>X14416.1/171-331
AUACUUACCUGGACGG-GGUCU-AUGG---GCGAUCAU-GUAG-GUCCA-UGGCCU---AGG-UUGGUGACUUUCAUU-----GCACUU-UGGAGG-GGUGCCCG---------CCUA-AGAUCAGCCCAA---GAGG--UUG-AGUCU-
ACAUCAUAAUUUGUUGCUG---AGGGG-GCAUGCGUU--CGCGCAGCCCCUGC
>X14415.1/175-335
UUACUUACCUGGACGG-GGUCA-AUUG---GCGAUCAU-GAAG-GUCCA-UGGCCU---AGG-UUGGUAACCUCCAUU-----GCACUU-AGGAGA-GGUGCCUA---------CCUA-AGAUCGGCCCAA---GUGG--CCG-AAUCU-
```

测试数据：

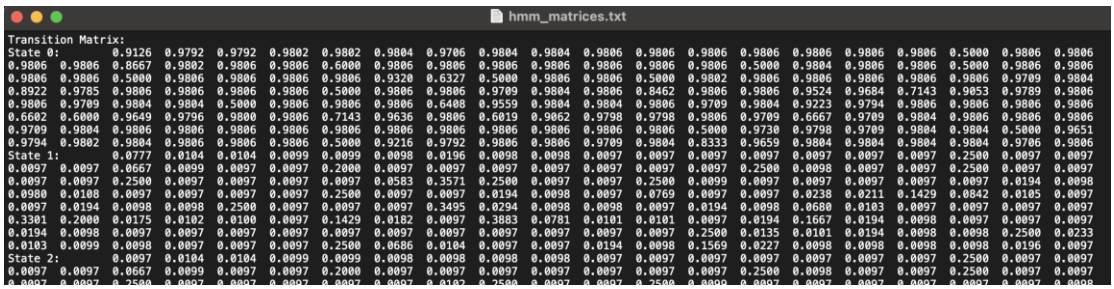
test.txt

```
AUACUUACCUGGACGGGGUCAAUGGGCGAUCAUGAACACCCAUGGCCUAGGUUGGUGACCAUCAUUGCACUUUGAAGGGGUGCCCGCCUAAGGUCGGCCCAAGUGGUCGAGCCUACGUCAUAAUUUGUUGCUGAGGGGGCCUGC
GUUCGCGCGGCCCCUGC
ACUUACCUGGCAGGGGAGAUACCAUGAUCACGAAGGUGGUUUUCCCAGGGCGAGGCUUAUCCAUUGCACUCCGGAUGUGCUGACCCCUGCGAUUUCCCCAAAUGUGGGAAACUCGACUGCAUAAUUUGUGGUAGUGGGGGACUG
CGUUCGCGCUUUCCCCUG
AUACUUACCUGGCAGGGGAGACACCAUGAUCAGGCAGGUGGUUUUCCCAGGGCGAGGCUCAUCCCCUGCACUCCGGGUGUGCUGACCCCUGCGAUUUCCCCAAAUGCGGGAAACUCGACUGCAUAAUUUGUGGUAGUGGGGGAC
UGCGUUUCGCGCUCUCCCCUG
```

构建的转移矩阵与发射矩阵：

hmm_matrices.txt

```
Emission Matrix (Match States):
M1:     0.8969  0.1340  0.9608  0.0196  0.0194  0.0291  0.9612  0.0194  0.0192  0.0192  0.0096  0.0192  0.2308  0.3846  0.0192  0.2019  0.0577  0.1442  0.3462  0.0673
0.5673  0.3529  0.5481  0.0577  0.1250  0.3942  0.0288  0.0385  0.9327  0.0192  0.0192  0.9519  0.2885  0.0388  0.9038  0.9231  0.0288  0.1250  0.0769  0.0192  0.0481
0.2404  0.1250  0.0769  0.0401  0.0096  0.0096  0.0202  0.6923  0.3750  0.1250  0.0288  0.0686  0.0577  0.6250  0.1346  0.0577  0.0577  0.2019  0.0485  0.4078  0.0532
0.0865  0.0096  0.8558  0.0096  0.0096  0.0096  0.0096  0.9615  0.0097  0.0577  0.0673  0.1250  0.0385  0.0865  0.2188  0.0291  0.0729  0.1042  0.0192  0.0096  0.0577
0.0583  0.6408  0.2115  0.0769  0.0385  0.0096  0.2692  0.0870  0.1942  0.0194  0.6923  0.0192  0.0583  0.2308  0.1633  0.0096  0.0096  0.0288  0.6538  0.9038  0.8732
0.0172  0.0202  0.0198  0.0385  0.1154  0.2143  0.0673  0.5096  0.6462  0.8400  0.0400  0.0192  0.0192  0.1359  0.6827  0.0485  0.1442  0.0192  0.0192  0.7500  0.0291
0.9135  0.8942  0.0096  0.0096  0.0096  0.0673  0.0096  0.0096  0.0769  0.0096  0.7885  0.0192  0.0133  0.1200  0.0288  0.0583  0.0583  0.1650  0.5287  0.0816  0.0686
0.0194  0.0481  0.0192  0.0192  0.0385  0.0388  0.0206  0.0385  0.0481  0.0096  0.0680  0.1262  0.0112  0.0097  0.0194  0.0097  0.0194  0.1553  0.1165
M2:     0.0103  0.0103  0.0098  0.9510  0.0194  0.0097  0.0097  0.9515  0.9231  0.0096  0.0096  0.0096  0.7212  0.2404  0.1923  0.0769  0.0385  0.0192  0.0096  0.2308
0.0769  0.1863  0.1250  0.5481  0.5192  0.0385  0.2692  0.0096  0.0192  0.0192  0.9615  0.0192  0.3942  0.0777  0.0577  0.0288  0.0192  0.0096  0.4135  0.2500  0.2885
0.0865  0.0577  0.2885  0.3462  0.7788  0.8365  0.6970  0.0308  0.0096  0.0192  0.0096  0.4216  0.1250  0.0288  0.0096  0.0096  0.5096  0.2115  0.4563  0.2427  0.1383
0.7596  0.9231  0.1154  0.0865  0.0096  0.0096  0.9423  0.0096  0.9515  0.1154  0.3750  0.5865  0.0096  0.0481  0.2396  0.0777  0.0521  0.0625  0.0481  0.5288  0.2308
0.1553  0.1068  0.5288  0.8365  0.7788  0.4615  0.0481  0.0145  0.6408  0.0583  0.0192  0.1346  0.2427  0.1154  0.5204  0.8558  0.8846  0.9038  0.0481  0.0385  0.0141
0.1379  0.0101  0.2178  0.0288  0.0192  0.0893  0.1154  0.0192  0.1385  0.0400  0.6800  0.2596  0.8654  0.0194  0.0096  0.6408  0.2212  0.0192  0.7212  0.0096  0.0777
0.0192  0.0192  0.0192  0.0096  0.0096  0.1442  0.0096  0.0096  0.1154  0.2404  0.0288  0.0096  0.1867  0.1800  0.0865  0.0097  0.0097  0.0097  0.2759  0.6939  0.0980
0.0485  0.8365  0.0865  0.0673  0.0192  0.8447  0.0103  0.8269  0.0192  0.8558  0.1942  0.1942  0.3596  0.8058  0.8932  0.8641  0.6117  0.1359  0.3301
M3:     0.0103  0.0206  0.0196  0.0098  0.0097  0.0194  0.0097  0.0097  0.0288  0.0096  0.9615  0.9519  0.0192  0.2404  0.6827  0.6923  0.8942  0.8173  0.3462  0.5673
0.0769  0.0882  0.0865  0.2788  0.3365  0.4423  0.0096  0.9327  0.0385  0.0192  0.0096  0.0096  0.1346  0.7670  0.0096  0.0096  0.8750  0.8269  0.0865  0.5192  0.6058
0.1442  0.1154  0.2981  0.2596  0.0288  0.0288  0.0101  0.2308  0.5577  0.8173  0.8558  0.1961  0.5769  0.2596  0.8269  0.7019  0.3269  0.0673  0.0194  0.1845  0.2979
```

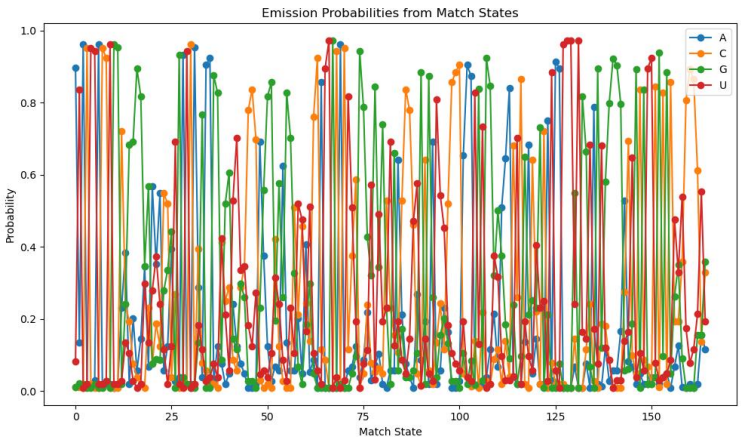


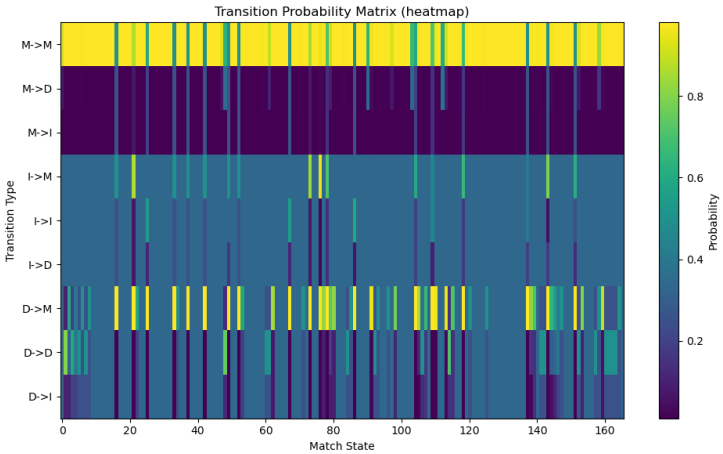
hmm_matrices.txt

```
Transition Matrix:
State 0:        0.9126  0.9792  0.9792  0.9802  0.9802  0.9804  0.9706  0.9804  0.9804  0.9806  0.9806  0.9806  0.9806  0.9806  0.9806  0.9806  0.5000  0.9806  0.9806
0.9806  0.9806  0.8667  0.9802  0.9806  0.9806  0.6000  0.9806  0.9806  0.9806  0.9806  0.9806  0.9806  0.9806  0.5000  0.9804  0.9806  0.9806  0.5000  0.9806  0.9806
0.9806  0.9806  0.5000  0.9806  0.9806  0.9806  0.9806  0.9320  0.6327  0.5000  0.9806  0.9806  0.5000  0.9802  0.9806  0.9806  0.9806  0.9806  0.9806  0.9709  0.9804
0.8922  0.9785  0.9806  0.9806  0.9806  0.9806  0.5000  0.9806  0.9806  0.9709  0.9804  0.9806  0.8462  0.9806  0.9806  0.9524  0.9684  0.7143  0.9053  0.9789  0.9806
0.9806  0.9709  0.9804  0.9804  0.5000  0.9806  0.9806  0.9806  0.6408  0.9559  0.9804  0.9804  0.9806  0.9709  0.9804  0.9223  0.9794  0.9806  0.9806  0.9806  0.9806
0.6602  0.6000  0.9649  0.9796  0.9800  0.9806  0.7143  0.9636  0.9806  0.6019  0.9062  0.9798  0.9798  0.9806  0.9709  0.6667  0.9709  0.9804  0.9806  0.9806  0.9806
0.9709  0.9804  0.9806  0.9806  0.9806  0.9806  0.9806  0.9806  0.9806  0.9806  0.9806  0.9806  0.9806  0.5000  0.9730  0.9798  0.9709  0.9804  0.9804  0.5000  0.9651
0.9794  0.9802  0.9804  0.9806  0.9806  0.9806  0.5000  0.9216  0.9792  0.9806  0.9806  0.9709  0.9804  0.8333  0.9659  0.9804  0.9804  0.9804  0.9804  0.9706  0.9806
State 1:        0.0777  0.0104  0.0104  0.0099  0.0099  0.0098  0.0196  0.0098  0.0098  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.2500  0.0097  0.0097
0.0097  0.0097  0.0667  0.0099  0.0097  0.0097  0.2000  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.2500  0.0098  0.0097  0.0097  0.2500  0.0097  0.0097
0.0097  0.0097  0.2500  0.0097  0.0097  0.0097  0.0097  0.0583  0.3571  0.2500  0.0097  0.0097  0.2500  0.0099  0.0097  0.0097  0.0097  0.0097  0.0097  0.0194  0.0098
0.0980  0.0108  0.0097  0.0097  0.0097  0.0097  0.2500  0.0097  0.0097  0.0194  0.0098  0.0097  0.0769  0.0097  0.0097  0.0238  0.0211  0.1429  0.0842  0.0105  0.0097
0.0097  0.0194  0.0098  0.0098  0.2500  0.0097  0.0097  0.0097  0.3495  0.0294  0.0098  0.0098  0.0097  0.0194  0.0098  0.0680  0.0103  0.0097  0.0097  0.0097  0.0097
0.3301  0.2000  0.0175  0.0102  0.0100  0.0097  0.1429  0.0182  0.0097  0.3883  0.0781  0.0101  0.0101  0.0097  0.0194  0.1667  0.0194  0.0098  0.0097  0.0097  0.0097
0.0194  0.0098  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.2500  0.0135  0.0101  0.0194  0.0098  0.0098  0.2500  0.0233
0.0103  0.0099  0.0098  0.0097  0.0097  0.0097  0.2500  0.0686  0.0104  0.0097  0.0097  0.0194  0.0098  0.1569  0.0227  0.0098  0.0098  0.0098  0.0098  0.0196  0.0097
State 2:        0.0097  0.0104  0.0104  0.0099  0.0099  0.0098  0.0098  0.0098  0.0098  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.2500  0.0097  0.0097
0.0097  0.0097  0.0667  0.0099  0.0097  0.0097  0.2000  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.0097  0.2500  0.0098  0.0097  0.0097  0.2500  0.0097  0.0097
```

其中 M1234 分别对应 ACGU ；State0～8 分别对应 M→M；M→D；M→I；I→M；I→I；I→D；D→M；D→D；D→I。

完整输出结果可见 hmm_profile_part2 文件夹中 output_files 中 hmm_matrices.txt 文件，下面是基于转移矩阵与发射矩阵画出的两个可视化的折线图和热图：





输出 log-likelihood 打分结果：

result.txt

```
0        AUACUUACCUGGACGGGGUCAAUGGGCGAU  -149.7307429214523
1        ACUUACCUGGCAGGGGAGAUACCAUGAUCA  -77.06794756831938
2        AUACUUACCUGGCAGGGGAGACACCAUGAU  -89.99352086286206
```

我们可以看到三条序列的 log-likelihood 数值均在-100 左右，代表它们在当前训练得到的 Profile-HMM 下具有较高的匹配可能性，属于相对合理对齐且同源性较高的序列，这说明我们的模型估计结果是合理的，代码实现具有一定的可信度。

6. 算法时间复杂度的估算与验证

（1）对于第一部分的时间复杂度分析：

这部分代码的主要功能包括构建拓扑结构，初始化发射和转移概率，使用前向后向算法计算状态概率，使用 Baum-Welch 算法训练参数，最后使用 Viterbi 算法进行对测试序列进行对齐。

主要函数中，create_topology()用于创建 HMM 的状态集合，时间复杂度为 O(L)；initialize_parameters() 根据 MSA 的 L 个匹配列，统计非 gap 字符频率，初始化发射概率，同时构造状态转移矩阵，时间复杂度为 O(N*L*Y)；forward_algorithm() 和 backward_algorithm()分别实现标准的前向与后向算法，外层循环为序列长度 L，内层循环为状态数≈3L，时间复杂度为 O(L*L^2)=O(L^3)；gamma_S() 用于计算后验概率和状态转移联合概率，为双层嵌套循环，时间复杂度为 O(L^3)；learning_algorithm() 使用 Baum-Welch

算法，使用 N 条训练序列的 gamma 进行更新，时间复杂度为 O(N*L*L^2 + N*L*L*Y)；update_matrices()对零概率平滑处理并重新归一化，最多遍历所有状态，时间复杂度为 O(L^2)；viterbi_algorithm() 实现 Viterbi 算法，使用动态规划，时间复杂度为 O(L*L^2) =O(L^3)。

所以代码总时间复杂度为 **O(N*L^3+N*L^2*Y)**。

如果用测试数据输入的话，理论运行时间是 N*L^3+N*L^2*Y≈5*10^3+5*10^2*4≈7000，按一秒可以进行 10^6 次运算计算，预计运行时间为 7 毫秒，而实际用时为 3.18 秒，我们认为这主要是因为代码实现中存在大量的非数值型操作（比如字符串拼接、列表动态操作、路径枚举等），这些虽然是常数时间复杂度，但是在实际运行中会大幅拉长处理时间因此导致了运行时间的略长。

```
● (base) celina@bogon 3 % python /Users/celina/Desktop/3/hmm_profile.py
  testcase
  GATGTCAT   -G-ATGTCAT

  ⏱程序运行时间： 3.18 秒 _
```

而在我们的 U1 spliceosomal RNA 训练数据中，序列数 N=81，序列长度 L 大约 160－180 bp，N*L^3+N*L^2*Y≈100*200^3+100*200^2*4≈10^9 次操作量，按一秒可以进行 10^6 次运算计算，预计运行时间为 1000 秒，实际时间则会比这更长，所以无法较快得到结果。

（2）对于第二部分的时间复杂度分析：

第二部分代码的主要函数中，boolify()用于生成二维布尔矩阵，标记 MSA 中每个字符是否是 gap，时间复杂度 O(N*L)；calc_match_states()判断每一列是否是“match state”，时间复杂度为 O(L*N)；calc_emissons()计算发射概率，按列/状态分类统计字母频数，然后进行归一化，时间复杂度为 O(L*N*Y)；calc_transmissions()计算各状态之间的转移概率，遍历每对相邻列，统计状态转移类型，外层循环 L 次，每次循环统计 N 个序列状态组合，时间复杂度为 O(N*L)；_viterbi()使用 JIT 优化，加快运行速度，实现标准 viterbi 算法的动态规划，外层循环为测试序列长度 L，内层循环为 HMM 序列长度 L，时间复杂度为 O(L^2)。

所以代码总时间复杂度为 **O(L^2+ L*N*Y)**。

在我们的 U1 spliceosomal RNA 训练数据中，序列数 N=81，序列长度 L 大约 160－180 bp，L^2+ L*N*Y≈200^2+200*100*4≈1.2*10^6 次操作量，按一秒可以进行 10^6 次运算计算，预计运行时间为 1 秒，而 Numba 首次编译需耗时 1－2 秒，总理论耗时为 2-3 秒。而在实际运行中耗时为 6.23 秒，说明之前时间复杂度的估计合理。

```
● (base) celina@bogon 生信大作业 % python main.py --train-data train.txt --test-data test.txt --out result.txt
  [Info] Writing emission matrix to: hmm_matrices.txt
  [Success] Emission and transition matrices written to: hmm_matrices.txt
  [Info] Emission matrix plot saved as emissions_plot.png
  Transition matrix heatmap saved as transitions_heatmap.png
  [2 of 0] in progress..

  ⏱程序运行时间： 6.23 秒 _
```

参考文献：

[1] Eddy SR. Profile hidden Markov models. *Bioinformatics*. 1998;14(9):755-763. doi:10.1093/bioinformatics/14.9.755

[2] Protein modeling using hidden Markov models: analysis of globins. IEEE Conference Publication | IEEE Xplore. https://ieeexplore.ieee.org/document/270611

[3] Pattabiraman S, Warnow T. Profile Hidden Markov Models Are Not Identifiable. *IEEE/ACM Trans Comput Biol Bioinform*. 2021;18(1):162-172. doi:10.1109/TCBB.2019.2933821
[4] Durbin, R., Eddy, S., Krogh, A. and Mitchison, G. (1998) Biological sequence analysis: Probabilistic models of proteins and nucleic acids. Cambridge University Press.